

HelixTranslate

HelixTranslate

सत्यापित-मॉडल पुस्तक अनुवाद — डिज़ाइन से ईमानदार, कभी मूक विकल्प नहीं।

सारांश

HelixTranslate एक उच्च-प्रदर्शन वाला, Go-आधारित ईबुक अनुवाद प्लेटफॉर्म है जो सत्यापित LLM प्रदाताओं का उपयोग करके 100 से अधिक भाषाओं के बीच पुस्तकों का अनुवाद करता है। इसमें वास्तविक समय में WebSocket निगरानी और एक सख्त “मूक-विकल्प नहीं” रूटिंग नीति है, जो चुपचाप गुणवत्ता गिराने के बजाय खुले तौर पर विफल होती है।

संक्षिप्त विवरण

Go-आधारित सार्वभौमिक ईबुक अनुवाद टूलकिट। यह FB2, EPUB, TXT, HTML, PDF और DOCX प्रारूपों का 100 से अधिक भाषाओं में अनुवाद करता है, जिसमें सत्यापित LLM (LLMsVerifier ब्रिज के माध्यम से) का उपयोग होता है। इसमें REST/HTTP-3 और gRPC API, वितरित प्रसंस्करण और वास्तविक समय में WebSocket निगरानी डैशबोर्ड शामिल है।

विस्तृत विवरण

HelixTranslate एक उद्यम-स्तरीय, Go-आधारित प्रणाली है जो संपूर्ण पुस्तकों का अनुवाद करने के लिए LLM प्रदाताओं का उपयोग करती है — न कि केवल पैराग्राफ़ या अंशों का, बल्कि संपूर्ण पुस्तक-लंबाई के कार्यों का, आरंभ से अंत तक। यह कई ईबुक प्रारूपों (FB2, EPUB, TXT, HTML, PDF, DOCX) को पार्स और पुनर्निर्मित करता है, 100 से अधिक भाषाओं का समर्थन करता है (स्वचालित पहचान के साथ), और CLI टूल्स तथा API सर्वर (REST पर HTTP/3, gRPC और WebSocket इवेंट स्ट्रीम) प्रदान करता है, ताकि यह टर्मिनल वर्कफ़्लो या सेवा जाल दोनों में समान रूप से फिट हो सके। इसकी विशेषता यह है कि यह मॉडल का चयन कैसे करता है: किसी प्रदाता को हार्डकोड करके यह आशा करने के बजाय कि वह स्वस्थ रहेगा, HelixTranslate सभी मॉडल प्राधिकरण को LLMsVerifier ब्रिज (pkg/bridge) को सौंपता है, जो सबसे मज़बूत सत्यापित API मॉडल का चयन करता है और एक निर्धारक, स्कोर-क्रमित विकल्प श्रृंखला लौटाता है। मॉडल की पात्रता प्रतिक्रिया क्षमता, कोड, सुविधाओं की समृद्धि और विश्वसनीयता के

भारित स्कोर के आधार पर तय की जाती है — इसलिए आपके अनुवाद को करने वाला मॉडल अपनी जगह कॉन्फ़िगरेशन फ़ाइल में दिखने के बजाय यह साबित करके कमाता है कि वह काम करता है।

महत्वपूर्ण बात यह है कि प्रणाली कोड में ही “मूक विकल्प नहीं” का आदेश लागू करती है: यदि कोई प्रदाता API कुंजी मौजूद नहीं है, या कोई ऑपरेटर स्पष्ट रूप से अनुपलब्ध प्रदाता का अनुरोध करता है, तो पाइपलाइन एक ईमानदार हार्ड एरर लौटाती है, बजाय इसके कि चुपचाप प्रदाताओं को बदल दिया जाए या स्थानीय रनटाइम पर उतरकर सब कुछ ठीक होने का दिखावा किया जाए — यह नियम एक समर्पित प्री-बिल्ट गेट और एक संबद्ध म्यूटेशन टेस्ट द्वारा सुनिश्चित किया गया है। स्थानीय रनटाइम (Ollama, llama.cpp) को जानबूझकर डिफ़ॉल्ट पथ से हटा दिया गया है ताकि कोई कमज़ोर इंजन कभी सत्यापित मॉडल की जगह चुपचाप न ले सके। अनुवाद कोर के चारों ओर वास्तविक समय में WebSocket निगरानी उपतंत्र स्थित है: अनुवाद CLI टाइप्ड इवेंट्स को एक निगरानी सर्वर पर भेजता है, जो एक लाइव वेब डैशबोर्ड को संचालित करता है, जबकि दूरस्थ SSH वर्क्स कार्यभार को वितरित अनुवाद के लिए फैला देते हैं। इसके ऊपर बहु-चरणीय परिष्करण (संगति के लिए), तैयारी-चरण गुणवत्ता विश्लेषण, अनुवाद कैशिंग (लंबे इनपुट पर लागत नियंत्रण के लिए), और दृष्टि-आधारित गुणवत्ता आश्वासन की परतें हैं। संपूर्ण प्लेटफ़ॉर्म एक “धोखाधड़ी-विरोधी इंजीनियरिंग संविधान” का पालन करता है: परीक्षणों को वास्तविक, उपयोगकर्ता-दृश्य परिणाम साबित करने होते हैं, जिन्हें केवल हरे चेकमार्क के बजाय अनिवार्य म्यूटेशन परीक्षण द्वारा समर्थित किया जाता है।

सामग्री

हमने इसे क्यों बनाया

लंबी-प्रपत्र पुस्तकों का विश्वसनीय और ईमानदार अनुवाद करना — कभी भी “गुणवत्ता में गिरावट लेकिन मौजूद” अनुवाद को स्वीकार न करना। डिज़ाइन का मूल सिद्धांत यह है कि अनुपलब्ध या अप्रमाणित अनुवाद एक ज़ोरदार, कठोर त्रुटि होनी चाहिए, और मॉडल चयन हमेशा किसी वास्तविक रूप से सत्यापित प्रदाता पर आधारित होना चाहिए, न कि किसी पूर्वनिर्धारित अनुमान या चुपचाप स्थानीय विकल्प पर निर्भर रहने पर।

यह क्यों क्रांतिकारी है

अधिकांश LLM अनुवाद पाइपलाइन चुपचाप विफल हो जाती हैं — वे चुपके से कमज़ोर मॉडल पर लौट आती हैं, स्थानीय रनटाइम पर उतर जाती हैं, या आंशिक आउटपुट देती हैं जबकि टेस्ट सूट अभी भी हरा दिखता है और गुणवत्ता में गिरावट की ओर किसी का ध्यान नहीं जाता। HelixTranslate इस संपूर्ण विफलता मोड को संरचनात्मक रूप से असंभव बना देता है: मॉडल चयन सत्यापन द्वारा नियंत्रित होता है, फॉलबैक श्रृंखला निर्धारक और पूरी तरह पारदर्शी होती है, और “कोई कुंजी नहीं / कोई सत्यापित मॉडल नहीं” की स्थिति एक ईमानदार कठोर त्रुटि में बदल जाती है, चुपचाप कंधे उचकाने में नहीं। इस एकल डिज़ाइन निर्णय से “क्या यह अनुवाद वास्तव में किसी सक्षम, सत्यापित मॉडल पर चला?” का सवाल एक ऐसी आशा से बदल जाता है जिसे आप जाँच नहीं सकते, एक ऐसी गारंटी में जिसे सिस्टम आपके लिए लागू करता है।

क्या है नवाचारी

- सत्यापन-नियंत्रित मॉडल रूटिंग LLMsVerifier ब्रिज के माध्यम से — सबसे मज़बूत सत्यापित मॉडल स्वचालित रूप से चुना जाता है, जिससे ऑपरेटर केवल इरादा व्यक्त करते हैं, विक्रेता के नाम नहीं, और कभी भी ऐसे प्रदाता का चयन नहीं करते जो उपलब्ध न हो।
- कोड में लागू नो-साइलेंट-फॉलबैक गारंटी — चार स्पष्ट रूटिंग शाखाएँ (मॉक / स्पष्ट-सत्यापक / स्पष्ट-प्रदाता / ब्रिज-डिफ़ॉल्ट), जिनमें से प्रत्येक चुपचाप स्विच करने के बजाय कठोर त्रुटि उत्पन्न करती है, साथ ही डिफ़ॉल्ट पथ से स्थानीय रनटाइम को जानबूझकर हटा दिया गया है ताकि कमज़ोर विकल्प के लिए कोई फॉलबैक न हो।
- यांत्रिक प्रवर्तन — CM-NO-LOCAL-RUNTIME प्री-बिल्ड गेट और एक संबद्ध म्यूटेशन टेस्ट, जो बिल्ड के समय यह सुनिश्चित करता है कि डिफ़ॉल्ट पथ पर कभी भी स्थानीय-रनटाइम क्लाइंट का निर्माण न हो: गारंटी कभी कमज़ोर नहीं हो सकती क्योंकि अगर ऐसा होता है तो बिल्ड विफल हो जाता है।
- निर्धारक, स्कोर-क्रमित फॉलबैक श्रृंखला — सत्यापित मॉडलों के बीच प्रदाता-से-प्रदाता फेलओवर की अनुमति है और पूरी तरह पारदर्शी है, जो निषिद्ध चुपचाप फॉलबैक से एक सिद्धांतगत अंतर है: आपको हमेशा पता रहता है कि किस सक्षम मॉडल ने काम संभाला।
- रीयल-टाइम WebSocket मॉनिटरिंग — टाइपड अनुवाद घटनाएँ लाइव डैशबोर्ड पर स्ट्रीम की जाती हैं, जिसमें वितरित SSH वर्कसे होते हैं ताकि पुस्तक-लंबाई का कार्य दिखाई दे और समानांतर हो, न कि किसी काले बक्से की तरह।
- एंटी-ब्लफ़ परीक्षण व्यवस्था — म्यूटेशन परीक्षण, नकारात्मक अभिकथन, वास्तविक-सिस्टम रन और विज्ञान-चालित QA मिलकर यह सुनिश्चित करते हैं कि “टेस्ट पास” कभी चुपचाप यह छिपा न सके कि “फीचर वास्तव में काम नहीं कर रहा।”

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- ईमानदार अनुवाद पाइपलाइन की गारंटी (कोई चुपचाप गुणवत्ता गिरावट नहीं)। इसका समाधान LLMsVerifier ब्रिज में सभी मॉडल प्राधिकरण को केंद्रीकृत करके किया गया, ताकि एक ही निर्णय बिंदु हो जिसे नियंत्रित किया जा सके, चार स्पष्ट रूटिंग शाखाओं को कोडित किया गया जो अनुमान लगाने के बजाय ज़ोरदार त्रुटि उत्पन्न करती हैं, डिफ़ॉल्ट पथ से स्थानीय-रनटाइम फॉलबैक को पूरी तरह हटा दिया गया, और इस नियम को बिल्ड गेट और म्यूटेशन टेस्ट के साथ मज़बूती से लागू किया गया जो गारंटी हटाए जाने पर बिल्ड को विफल कर देता है।
- “टेस्ट पास, फीचर टूटा हुआ।” संविधान इस विफलता मोड को स्पष्ट रूप से नामित करता है और एंटी-ब्लफ़ परीक्षण व्यवस्था के माध्यम से इसे परास्त करता है: कार्यान्वयन की तुच्छताओं के बजाय ठोस उपयोगकर्ता-दृश्य अभिकथन, वास्तविक सिस्टम का उपयोग (मॉक केवल यूनिट टेस्ट तक सीमित), अनिवार्य म्यूटेशन परीक्षण (जानबूझकर फीचर तोड़ें और टेस्ट लाल होना चाहिए), और विज्ञान-सत्यापित QA जो वास्तव में आउटपुट की जाँच करता है।
- लंबी-प्रपत्र, बहु-प्रारूप गुणवत्ता। पुस्तक-लंबाई के इनपुट स्थिरता और बजट दोनों पर दबाव डालते हैं; इसका समाधान मल्टी-पास पॉलिशिंग से किया गया जो पाठ की पुनः समीक्षा करता है, तैयारी चरण में कार्य का आकार निर्धारित करता है, और अनुवाद कैंशिंग से एक ही अंश के लिए दोबारा भुगतान से बचाता है।

तकनीकी स्टैक

- **Go** — इसे इसके समवर्ती प्रिमिटिव्स के लिए चुना गया है, जो एक साथ कई अध्यायों के पार्सिंग, अनुवाद और स्ट्रीमिंग के लिए स्वाभाविक रूप से उपयुक्त हैं; उच्च-समवर्ती बैकएंड, मॉड्यूल `digital.vasic.translator`।
- **Gin** — REST API इंटरफ़ेस को परोसने के लिए एक तेज़, न्यूनतम HTTP राउटर के रूप में चुना गया।
- **QUIC / HTTP/3 (quic-go)** — REST API को कम-विलंबता, आधुनिक ट्रांसपोर्ट प्रदान करने के लिए चुना गया, जो अपूर्ण नेटवर्क पर भी स्थिर रहता है।
- **gRPC + Protocol Buffers** — REST के साथ-साथ प्रोग्रामेटिक कॉलर्स के लिए एक सशक्त-टाइप, उच्च-प्रदर्शन सेवा इंटरफ़ेस के रूप में चुना गया।
- **Gorilla WebSocket** — वास्तविक समय में टाइप अनुवाद-इवेंट स्ट्रीम को ले जाने के लिए चुना गया, जो मॉनिटरिंग डैशबोर्ड को लाइव फीड करता है।
- **PostgreSQL, SQLite, Redis** — एक सोची-समझी तीन-स्तरीय संरचना: PostgreSQL टिकाऊ रिलेशनल डेटा के लिए, SQLite एम्बेडेड/स्थानीय स्थिति के लिए (यह ब्रिज के सत्यापित-मॉडल स्टोर `data/verified_models.db` को भी सपोर्ट करता है), और Redis हॉट कैश के रूप में।
- **unidoc/unioffice + unipdf** — कठिन फॉर्मेट्स को संभालने के लिए चुना गया: DOCX और PDF पार्सिंग और पुनर्निर्माण, ताकि मल्टी-फॉर्मेट ईबुक्स बिना किसी बदलाव के वापस आएँ।
- **Cobra** — unified-translator और इसके सहायक टूल्स को शक्ति प्रदान करने वाले CLI फ्रेमवर्क के रूप में चुना गया।
- **golang-jwt (JWT HS256)** — स्टेटलेस API प्रमाणीकरण के लिए चुना गया, जिसे प्रति-आईपी टोकन-बकेट रेट लिमिटिंग और TLS/QUIC ट्रांसपोर्ट सुरक्षा के साथ जोड़ा गया है ताकि इंटरफ़ेस को मजबूत बनाया जा सके।
- **LLMsVerifier ब्रिज (pkg/bridge)** — मुख्य कड़ी: सबसे सशक्त सत्यापित मॉडल और उसकी नियतात्मक फॉलबैक श्रृंखला का स्रोत, और नो-साइलेंट-फॉलबैक गारंटी का एकमात्र प्रवर्तन बिंदु।
- **Testify** — Go टेस्ट सूट के लिए चुना गया, जिसमें समर्पित `provider_routing_test.go` और म्यूटेशन गेट्स शामिल हैं जो ईमानदारी के नियमों को कायम रखते हैं।
- **Docker / Podman (रूटलेस) + Compose** — कंटेनरीकृत, वितरित डिप्लॉयमेंट (`docker-compose.distributed.yml`) के लिए चुना गया, जिसमें रूटलेस Podman से सुरक्षा मुद्रा को और मजबूत किया गया है।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। कार्यात्मक प्लेटफ़ॉर्म; `VERSION/Makefile/AGENTS.md` में संस्करण असंगत रूप से दर्ज है, इसलिए इसे अस्थिर माना जाता है।
- **लाइसेंस:** अघोषित। `README` में MIT का दावा किया गया है, लेकिन इसे `LICENSE` फ़ाइल से सत्यापित नहीं किया गया है — दावा करने से पहले जाँच लें।
- **डैशबोर्ड/मॉनिटर** एंडपॉइंट केवल लोकलहोस्ट के लिए हैं, सार्वजनिक नहीं। दस्तावेज़ों में `WebSocket` प्रदर्शन संख्याएँ लक्ष्य के रूप में बताई गई हैं, सत्यापित नहीं। `ARCHITECTURE.md` में अभी भी हटाए गए Ollama/स्थानीय इंजन सूचीबद्ध हैं (पुराना डेटा)।

प्राथमिकता स्तर: Helix-प्राथमिक (LLM-इंफ्रास्ट्रक्चर क्लस्टर)। Helix प्लेटफ़ॉर्म परिवार में HelixTrack के बाद स्थान।